

MODULE

3

Working with Dates, Times, and Durations

Master every date, time, and duration function in M — from extracting year and month components to building production-ready Date dimension tables, computing business-day intervals, and handling time zones for global Power BI deployments.

Module Contents

■ Module Overview

■ Lesson 1: The M Date/Time Type System

- Four Temporal Types: date, time, datetime, datetimezone
- The Duration Type
- Literal Syntax for Each Type
- Type Conversions: Date.From, DateTime.From, DateTimeZone.From
- When to Use Each Type

■ Lesson 2: Date Functions

- Extracting Components: Date.Year, Date.Month, Date.Day
- Calendar Awareness: Date.DayOfWeek, Date.DayOfYear, Date.WeekOfYear, Date.WeekOfMonth
- Quarter Functions: Date.QuarterOfYear
- Month Name and Day Name: Date.MonthName, Date.DayOfWeekName
- Boundary Functions: Date.StartOfDay, Date.EndOfDay, Date.StartOfMonth, Date.EndOfMonth
- Date.StartOfQuarter, Date.EndOfQuarter, Date.StartOfYear, Date.EndOfYear
- Date.StartOfWeek, Date.EndOfWeek
- Arithmetic: Date.AddDays, Date.AddMonths, Date.AddYears, Date.AddQuarters, Date.AddWeeks
- Comparing Dates and Date.IsInCurrentMonth, Date.IsInCurrentYear
- Date.From and Parsing Dates from Text

■ Lesson 3: Time and DateTime Functions

- Time Components: Time.Hour, Time.Minute, Time.Second
- Time Arithmetic: Time.StartOfHour
- DateTime Functions: DateTime.Date, DateTime.Time
- DateTime.LocalNow and DateTime.FixedLocalNow
- DateTime.IsInCurrentHour, DateTime.IsInCurrentMinute
- Combining Date and Time: DateTime.From

■ Lesson 4: DateTimeZone Functions and Time Zones

- Why Time Zones Matter in Power BI
- DateTimeZone.LocalNow and DateTimeZone.UtcNow
- DateTimeZone.From, DateTimeZone.ToLocal, DateTimeZone.ToUtc
- DateTimeZone.RemoveZone, DateTimeZone.SwitchZone
- DateTimeZone.ZoneHours and DateTimeZone.ZoneMinutes
- Best Practices for Global Deployments

■ Lesson 5: Duration Functions

- The Duration Type and #duration Literal

- Duration.Days, Duration.Hours, Duration.Minutes, Duration.Seconds
- Duration.TotalDays, Duration.TotalHours, Duration.TotalMinutes, Duration.TotalSeconds
- Duration Arithmetic: Adding and Subtracting Durations
- Duration.From and Duration.ToRecord
- Computing Business Days and Working Hours

■ Lesson 6: Building a Production Date Dimension Table

- Why a Separate Date Table Is Required
- Generating Dates with List.Dates
- Adding All Required Columns
- Adding Fiscal Calendar Columns
- Adding Relative Date Columns
- Marking as Date Table in Power BI
- Complete Date Table M Code

■ Demonstration 1: Parsing and Standardising Mixed Date Formats

■ Demonstration 2: Building a Complete Date Dimension from Scratch

■ Lab 3: Dates, Times, and Durations

- Exercise 1: Extract Date Components and Build Calendar Columns
- Exercise 2: Duration Calculations — Lead Times and Ageing
- Exercise 3: Time Zone Handling for a Global Dataset
- Exercise 4: Build and Load a Complete Date Dimension Table

■ Module Review and Takeaways

- Review Questions
- Best Practices
- Common Issues and Solutions
- Key Functions Quick Reference

Module Overview

Every business intelligence system lives or dies by its ability to analyse data across time. Sales year-over-year, monthly trend lines, lead-time analysis, SLA compliance, ageing reports — all of these depend on correct, consistent handling of dates and times in the data model. Power Query M provides a rich, dedicated set of temporal functions covering four distinct types: **date**, **time**, **datetime**, and **datetimezone**, plus the **duration** type for representing time intervals.

Beyond individual functions, this module covers the single most important structural component of any Power BI model: the **Date dimension table**. Power BI's built-in time intelligence functions — **TOTALYTD**, **SAMEPERIODLASTYEAR**, **DATESINPERIOD**, and dozens more — only work correctly when a dedicated, contiguous Date table is present and marked correctly. You will build one entirely in M by the end of this module.

NOTE Module 3 builds on the type system knowledge from Module 1 and the conditional logic patterns from Module 2. You should be comfortable with `Table.AddColumn`, the `let...in` structure, and `if-then-else` before proceeding.

Before You Begin

- Complete Modules 1 and 2 and their associated lab exercises.
- Have Power BI Desktop open with the FactSales query from previous labs loaded.
- Download the lab dataset: GlobalSales.csv (contains Order Date, Ship Date, and UTC Timestamp columns in mixed formats) from the companion materials.
- Review your FactSales Applied Steps — you will be adding date-derived columns to it.

LESSON 1

The M Date/Time Type System

After completing this lesson, you will be able to:

- Name the five temporal types in M and describe what each represents.
- Write literal values for date, time, datetime, datetimezone, and duration.
- Convert between temporal types using `Date.From`, `DateTime.From`, and related functions.
- Explain the difference between `datetime` and `datetimezone`.
- Choose the correct temporal type for a given business scenario.

Five Temporal Types

M has five distinct temporal types. Choosing the wrong one is one of the most common sources of errors in Power Query, particularly when data arrives from systems in different time zones:

Type	Literal Example	Purpose and When to Use
<code>type date</code>	<code>#date(2024, 3, 15)</code>	Calendar date only. No time component. Represents a full calendar day. Use for: Order Date, Birth Date, Invoice Date, any column that is a date without a clock time.
<code>type time</code>	<code>#time(14, 30, 0)</code>	Time of day only. No date component. Hours (0-23), minutes (0-59), seconds (0-59 with fractional). Use for: shift start/end, opening hours, departure time.
<code>type datetime</code>	<code>#datetime(2024, 3, 15, 14, 30, 0)</code>	Combined date and time. No timezone offset. The most common type for timestamps from systems that record both date and time. Use for: order timestamps, log entries, event times when timezone is implied or irrelevant.
<code>type datetimezone</code>	<code>#datetimezone(2024, 3, 15, 14, 30, 0, 1, 0)</code>	DateTime with an explicit UTC offset (+HH:MM). Use for: data from global systems, API timestamps in ISO 8601 format (2024-03-15T14:30:00+01:00), any column where the timezone must be preserved for accurate comparison.
<code>type duration</code>	<code>#duration(1, 2, 30, 0)</code>	A time interval: days, hours, minutes, seconds. Not a point in time — a span. Use for: lead times, SLA durations, age calculations, elapsed time.

Literal Syntax

Each temporal type has a dedicated literal constructor. Understanding these prevents the common mistake of writing dates as text strings:

Temporal literal constructors:

```
// date literal – year, month, day
OrderDate = #date(2024, 3, 15) // March 15, 2024

// time literal – hours (24h), minutes, seconds
ShiftStart = #time(8, 30, 0) // 08:30:00
WithFrac = #time(14, 30, 45.5) // 14:30:45.5 (half second)

// datetime literal – year, month, day, hour, minute, second
OrderStamp = #datetime(2024, 3, 15, 14, 30, 0)

// datetimezone literal – year, month, day, hour, minute, second, tz_hours,
// tz_minutes
GlobalStamp = #datetimezone(2024, 3, 15, 14, 30, 0, 1, 0) // UTC+01:00
```

```
UTCStamp = #datetimezone(2024, 3, 15, 13, 30, 0, 0, 0) // UTC (same moment)

// duration literal – days, hours, minutes, seconds
OneWeek = #duration(7, 0, 0, 0) // 7 days
TwentyMins = #duration(0, 0, 20, 0) // 20 minutes
NinetyMins = #duration(0, 1, 30, 0) // 1 hour 30 minutes
```

WARNING Never store dates as text strings like '2024-03-15' in your data model. Text dates cannot participate in date relationships, DAX time intelligence functions, or Power BI's auto date/time hierarchy. Always cast to type date using `Table.TransformColumnTypes` or `Date.From`.

Type Conversion Functions

Converting between temporal types is a frequent task. Each conversion function accepts the closest compatible input type and can also parse text strings in standard ISO 8601 format:

Type conversion examples:

```
// Text to date – ISO 8601 format parsed automatically
Date.From("2024-03-15") // #date(2024, 3, 15)
Date.From("15/03/2024", "en-GB") // locale-aware parsing

// DateTime to date (drops time component)
Date.From(#datetime(2024, 3, 15, 14, 30, 0)) // #date(2024, 3, 15)

// Date to datetime (midnight time appended)
DateTime.From(#date(2024, 3, 15)) // #datetime(2024, 3, 15, 0, 0, 0)

// Text to datetime
DateTime.From("2024-03-15T14:30:00") // #datetime(2024, 3, 15, 14, 30, 0)

// DateTime to datetimezone (adds UTC offset)
DateTimeZone.From(#datetime(2024, 3, 15, 14, 30, 0))
// = #datetimezone(2024, 3, 15, 14, 30, 0, 0, 0) (UTC assumed)

// Number (OLE Automation date) to date
Date.From(45366) // #date(2024, 3, 15)
// Excel stores dates as integers – this converts them
```

LESSON 2

Date Functions

After completing this lesson, you will be able to:

- Extract year, month, day, weekday, and week number components from a date.
- Retrieve month name and day name with locale control.
- Find the start and end boundaries of days, weeks, months, quarters, and years.
- Add days, months, years, quarters, and weeks to a date using Date.Add* functions.
- Compare dates and use Date.IsInCurrentMonth and related functions.
- Parse dates from text strings with culture-aware formatting.

Extracting Date Components

The most frequently used date functions extract individual calendar components from a date value. These are the foundation of every Date dimension table:

Date.Year

Syntax	Date.Year(dateTime as any) as nullable number	
Parameter	Type	Description
dateTime	any	A date, datetime, or datetimezone value.
Returns	Returns the year component as an integer (e.g. 2024).	
Example	Date.Year(#date(2024, 3, 15))	
Result	2024	
NOTE	Accepts date, datetime, and datetimezone inputs interchangeably. Returns a number (not text) — use Text.From(Date.Year(...)) if you need a text year.	

Date.Month

Syntax	Date.Month(dateTime as any) as nullable number	
Parameter	Type	Description
dateTime	any	A date, datetime, or datetimezone value.

Returns	Returns the month as an integer 1–12.
Example	<code>Date.Month(#date(2024, 3, 15))</code>
Result	3
NOTE Returns a number, not a name. Use <code>Date.MonthName()</code> if you need 'March'. Use this for sorting — <code>MonthName</code> sorts alphabetically, <code>Month</code> sorts numerically.	

Date.Day

Syntax	<code>Date.Day(dateTime as any) as nullable number</code>	
Parameter	Type	Description
<code>dateTime</code>	<i>any</i>	A date, datetime, or datetimezone value.
Returns	Returns the day of the month as an integer 1–31.	
Example	<code>Date.Day(#date(2024, 3, 15))</code>	
Result	15	

Calendar Awareness Functions

Date.DayOfWeek

Syntax	Date.DayOfWeek(dateTime as any, optional firstDayOfWeek as nullable number) as nullable number	
Parameter	Type	Description
dateTime	any	A date, datetime, or datetimezone value.
firstDayOfWeek	nullable number	Optional. Day.Sunday (0), Day.Monday (1), etc. Default: Day.Sunday.
Returns	Returns the day of the week as an integer 0–6, where 0 = firstDayOfWeek.	
Example	Date.DayOfWeek(#date(2024, 3, 15), Day.Monday)	
Result	4 (Friday, since Monday=0)	
NOTE	The firstDayOfWeek parameter shifts the numbering. Use Day.Monday for ISO-standard week calculations (Monday=0, Sunday=6). The default Day.Sunday gives Sunday=0, Saturday=6.	

Date.DayOfWeekName

Syntax	<code>Date.DayOfWeekName(dateTime as any, optional culture as nullable text) as nullable text</code>
--------	--

Parameter		Type	Description
dateTime		any	A date, datetime, or datetimezone value.
culture		nullable text	Optional. Culture code e.g. 'en-US', 'fr-FR', 'de-DE'.
Returns	Returns the full name of the day of the week as localised text.		
Example	Date.DayOfWeekName(#date(2024, 3, 15), "en-US")		
Result	"Friday"		
NOTE Always specify a culture for production models to prevent locale-dependent results when reports are opened on machines with different regional settings.			

Date.DayOfYear		
Syntax	Date.DayOfYear(dateTime as any) as nullable number	
Parameter	Type	Description
dateTime	any	A date, datetime, or datetimezone value.
Returns	Returns the day number within the year, 1–366.	
Example	Date.DayOfYear(#date(2024, 3, 15))	
Result	75	
NOTE Useful for year-to-date (YTD) comparisons: filter rows where Date.DayOfYear([Date]) <= Date.DayOfYear(DateTime.Date(DateTime.LocalNow())).		

Date.WeekOfYear		Returns the ISO week number within the year (1-53). Pass Day.Monday for ISO 8601 week numbering.
Date.WeekOfYear(dateTime, optional firstDayOfWeek)		Date.WeekOfYear(#date(2024, 3, 15)) → 11
Date.WeekOfMonth		Returns the week number within the month (1-5).
Date.WeekOfMonth(dateTime, optional firstDayOfWeek)		Date.WeekOfMonth(#date(2024, 3, 15)) → 3
Date.QuarterOfYear		Returns the quarter as an integer 1-4. Combine with Text.From to produce 'Q1', 'Q2' etc.
Date.QuarterOfYear(dateTime)		Date.QuarterOfYear(#date(2024, 3, 15)) → 1
Date.MonthName		Returns the full localised month name. Always specify culture for consistent results.
Date.MonthName(dateTime, optional culture)		Date.MonthName(#date(2024, 3, 15), "en-US") → "March"

Boundary Functions

Boundary functions return the first or last moment of a calendar period. They are essential for calculating period-over-period comparisons and for building relative date filters:

Date.StartOfMonth

Syntax	Date.StartOfMonth(dateTime as any) as any	
Parameter	Type	Description
dateTime	any	A date, datetime, or datetimezone value.
Returns	Returns the first day of the month containing the given date.	
Example	Date.StartOfMonth(#date(2024, 3, 15))	
Result	#date(2024, 3, 1)	
NOTE	The return type matches the input type: if you pass a datetime, you get the first datetime of the month. If you pass a date, you get a date.	

Date.EndOfMonth

Syntax	Date.EndOfMonth(dateTime as any) as any	
Parameter	Type	Description
dateTime	any	A date, datetime, or datetimezone value.
Returns	Returns the last day of the month containing the given date. Handles leap years automatically.	
Example	Date.EndOfMonth(#date(2024, 2, 5))	
Result	#date(2024, 2, 29) (2024 is a leap year)	
NOTE	Date.EndOfMonth automatically computes the correct last day — 28, 29, 30, or 31 — making it far safer than calculating EndOfMonth = Date.StartOfMonth + 28/30/31.	

Date.StartOfDay	Returns midnight (00:00:00) of the given day.
Date.StartOfDay(dateTime, optional firstDayOfWeek)	Date.StartOfDay(#datetime(2024,3,15,14,30,0)) → #datetime(2024,3,15,0,0,0)
Date.EndOfDay	Returns the last moment of the given day.
Date.EndOfDay(dateTime, optional firstDayOfWeek)	Date.EndOfDay(#datetime(2024,3,15,14,30,0)) → #datetime(2024,3,15,23,59,59.9999999)
Date.StartOfQuarter	First day of the quarter containing the given date.
Date.StartOfQuarter(dateTime, optional firstDayOfWeek)	Date.StartOfQuarter(#date(2024,3,15)) → #date(2024,1,1)
Date.EndOfQuarter	Last day of the quarter containing the given date.

Date.EndOfQuarter(dateTime, optional firstDayOfWeek)	Date.EndOfQuarter(#date(2024,3,15)) → #date(2024,3,31)
Date.StartOfYear	First day of the year.
Date.StartOfYear(dateTime, optional firstDayOfWeek)	Date.StartOfYear(#date(2024,3,15)) → #date(2024,1,1)
Date.EndOfYear	Last day of the year.
Date.EndOfYear(dateTime, optional firstDayOfWeek)	Date.EndOfYear(#date(2024,3,15)) → #date(2024,12,31)
Date.StartOfWeek	First day of the week (Monday in this example).
Date.StartOfWeek(dateTime, optional firstDayOfWeek)	Date.StartOfWeek(#date(2024,3,15), Day.Monday) → #date(2024,3,11)
Date.EndOfWeek	Last day of the week containing the given date.
Date.EndOfWeek(dateTime, optional firstDayOfWeek)	Date.EndOfWeek(#date(2024,3,15), Day.Monday) → #date(2024,3,17)

Date Arithmetic Functions

Date arithmetic functions add or subtract calendar units to a date. They handle month-end edge cases automatically — adding one month to January 31 returns February 28/29, not March 2/3:

Date.AddDays		
Syntax	Date.AddDays(dateTime as any, numberOfDays as number) as any	
Parameter	Type	Description
dateTime	any	A date, datetime, or datetimezone value.
numberOfDays	number	Number of days to add. Use negative values to subtract.
Returns	Returns the date/datetime numberOfDays before or after the given value.	
Example	Date.AddDays(#date(2024, 3, 15), 10)	
Result	#date(2024, 3, 25)	
NOTE Date.AddDays correctly handles month and year boundaries. Date.AddDays(#date(2024, 3, 25), 10) = #date(2024, 4, 4). Use negative values to go backwards: Date.AddDays(today, -7) = one week ago.		
Date.AddMonths		Adds numberOfMonths to the date. Handles month-end clamping: Jan 31 + 1 month = Feb 28/29.
Date.AddMonths(dateTime, numberOfMonths)		Date.AddMonths(#date(2024, 1, 31), 1) → #date(2024, 2, 29) (Feb 29 in leap year)

Date.AddYears	Adds numberOfYears. Leap-year dates are clamped: Feb 29 in a leap year + 1 year = Feb 28.
Date.AddYears(dateTime, numberOfYears)	Date.AddYears(#date(2024, 2, 29), 1) → #date(2025, 2, 28) (2025 is not a leap year)
Date.AddQuarters	Adds numberOfQuarters (each quarter = 3 months).
Date.AddQuarters(dateTime, numberOfQuarters)	Date.AddQuarters(#date(2024, 1, 15), 2) → #date(2024, 7, 15)
Date.AddWeeks	Adds numberOfWeeks (each week = 7 days). Equivalent to Date.AddDays(date, n*7).
Date.AddWeeks(dateTime, numberOfWeeks)	Date.AddWeeks(#date(2024, 3, 15), 2) → #date(2024, 3, 29)

Relative Date Functions

These functions test whether a date falls within the current calendar period relative to today. They are useful for creating 'current period' filter flags in dimension tables:

Date.IsInCurrentDay	True if the date is today.
Date.IsInCurrentDay(dateTime)	Date.IsInCurrentDay(DateTime.Date(DateTime.LocalNow())) → true (if run today)
Date.IsInCurrentMonth	True if the date is in the current calendar month.
Date.IsInCurrentMonth(dateTime)	Date.IsInCurrentMonth(#date(2024, 3, 1)) → true (if run in March 2024)
Date.IsInCurrentQuarter	True if the date is in the current calendar quarter.
Date.IsInCurrentQuarter(dateTime)	Date.IsInCurrentQuarter(#date(2024, 1, 1)) → true (if run in Q1 2024)
Date.IsInCurrentYear	True if the date is in the current calendar year.
Date.IsInCurrentYear(dateTime)	Date.IsInCurrentYear(#date(2024, 6, 1)) → true (if run in 2024)
Date.IsInCurrentWeek	True if the date is within the current calendar week.
Date.IsInCurrentWeek(dateTime)	Date.IsInCurrentWeek(DateTime.Date(DateTime.LocalNow())) → true
Date.IsInPreviousMonth	True if the date is in the previous calendar month. Useful for MTD comparisons.
Date.IsInPreviousMonth(dateTime)	Date.IsInPreviousMonth(Date.AddMonths(DateTime.Date(DateTime.LocalNow()), -1)) → true
Date.IsInNextMonth	True if the date is in the next calendar month. Useful for forecasting filters.

```
Date.IsInNextMonth(dateTime)
```

```
Date.IsInNextMonth(Date.AddMonths(DateTime.Date(  
DateTime.LocalNow()), 1)) → true
```

**WAR
NING**

Date.IsInCurrentMonth and related functions evaluate relative to the time of query execution during refresh. In Power BI Service, this is the UTC time of the scheduled refresh. If your model is timezone-sensitive, use `DateTimeZone.UtcNow()` or `DateTimeZone.LocalNow()` explicitly rather than relying on the implicit 'current' in these functions.

LESSON 3

Time and DateTime Functions

After completing this lesson, you will be able to:

- Extract hour, minute, and second components from time and datetime values.
- Use DateTime.LocalNow and DateTime.FixedLocalNow appropriately.
- Separate the date and time components of a datetime value.
- Combine separate date and time values into a datetime.
- Use DateTime.IsInCurrentHour and related functions for real-time filtering.

Time Component Functions

Time component functions work identically on both type time and type datetime values. They extract the clock-time portion of the value:

Time.Hour	Returns the hour component (0-23). Works on time and datetime values.
Time.Hour(timeOrDateTime as any) as nullable number	Time.Hour(#time(14, 30, 45)) → 14
Time.Minute	Returns the minute component (0-59).
Time.Minute(timeOrDateTime as any) as nullable number	Time.Minute(#datetime(2024, 3, 15, 14, 30, 45)) → 30
Time.Second	Returns the second component including fractional seconds (0-59.9999).
Time.Second(timeOrDateTime as any) as nullable number	Time.Second(#time(14, 30, 45.75)) → 45.75
Time.StartOfHour	Returns the time value truncated to the start of the containing hour (minutes and seconds set to 0).
Time.StartOfHour(timeOrDateTime as any) as any	Time.StartOfHour(#time(14, 30, 45)) → #time(14, 0, 0)

DateTime Functions — Splitting and Combining

DateTime.Date

Syntax	DateTime.Date(dateTime as any) as nullable date	
Parameter	Type	Description
dateTime	any	A datetime or datetimezone value.
Returns	Extracts only the date portion, discarding the time. Returns a value of type date.	
Example	DateTime.Date(#datetime(2024, 3, 15, 14, 30, 0))	
Result	#date(2024, 3, 15)	
NOTE Use this when you need to join a datetime column to a date-type Date dimension table. The relationship will fail if one side is datetime and the other is date — DateTime.Date() fixes that mismatch.		

DateTime.Time		
Syntax	DateTime.Time(dateTime as any) as nullable time	
Parameter	Type	Description
dateTime	any	A datetime or datetimezone value.
Returns	Extracts only the time portion, discarding the date. Returns a value of type time.	
Example	DateTime.Time(#datetime(2024, 3, 15, 14, 30, 45))	
Result	#time(14, 30, 45)	
NOTE Use when you need to analyse patterns by time of day independently of the date — for example, grouping transactions by hour of the day across all dates.		

Current Date and Time Functions

Two functions return the current date and time at the moment of query evaluation. They behave very differently and choosing the wrong one can cause unexpected results in Power BI reports:

DateTime.LocalNow		
Syntax	DateTime.LocalNow() as datetime	
Parameter	Type	Description
Returns	Returns the current local date and time. Re-evaluated on every call during a single refresh.	
Example	DateTime.LocalNow()	
Result	#datetime(2024, 3, 15, 14, 30, 0) (at time of execution)	

NOTE `DateTime.LocalNow()` is evaluated lazily — each reference to it in your query may return a slightly different value. This matters in computed columns or conditional logic that compares two calls to `LocalNow`.

DateTime.FixedLocalNow

Syntax	DateTime.FixedLocalNow() as datetime	
Parameter	Type	Description
Returns	Returns the current local date and time, fixed to a single value for the entire query evaluation.	
Example	DateTime.FixedLocalNow()	
Result	#datetime(2024, 3, 15, 14, 30, 0) (same value for entire query)	
NOTE	DateTime.FixedLocalNow() returns the same timestamp for all references within a single query evaluation. Use this instead of DateTime.LocalNow() when you need consistent 'snapshot' time logic across multiple steps in the same query.	
TIP	Best practice: assign DateTime.FixedLocalNow() or DateTime.LocalNow() to a named step at the beginning of your query and reference that step name throughout. This ensures you are always comparing to the same reference time: RefreshTime = DateTime.FixedLocalNow(), then use RefreshTime in all subsequent comparisons.	

DateTime.IsInCurrent* Functions

Parallel to the `Date.IsInCurrent*` functions, `DateTime` provides finer-grained current-period tests:

DateTime.IsInCurrentHour	True if the datetime is within the current calendar hour.
<code>DateTime.IsInCurrentHour(dateTime)</code>	<code>DateTime.IsInCurrentHour(DateTime.LocalNow())</code> → true
DateTime.IsInCurrentMinute	True if the datetime is within the current calendar minute. Useful for near-real-time streaming.
<code>DateTime.IsInCurrentMinute(dateTime)</code>	<code>DateTime.IsInCurrentMinute(DateTime.LocalNow())</code> → true
DateTime.IsInCurrentSecond	True if the datetime is within the current second. Rarely used in batch ETL.
<code>DateTime.IsInCurrentSecond(dateTime)</code>	<code>DateTime.IsInCurrentSecond(DateTime.LocalNow())</code> → true
DateTime.IsInPreviousHour	True if the datetime falls in the hour immediately preceding the current one.

<code>DateTime.IsInPreviousHour(dateTime)</code>	<code>DateTime.IsInPreviousHour(Date.AddDays(DateTime.LocalNow(), 0)) → false (unless in previous hour)</code>
<code>DateTime.IsInNextHour</code>	True if the datetime falls in the hour immediately following the current one.
<code>DateTime.IsInNextHour(dateTime)</code>	<code>DateTime.IsInNextHour(Date.AddDays(DateTime.LocalNow(), 0)) → false (unless in next hour)</code>

LESSON 4

DateTimeZone Functions and Time Zones

After completing this lesson, you will be able to:

- Explain why time zones are critical in global Power BI deployments.
- Use `DateTimeZone.LocalNow` and `DateTimeZone.UtcNow` to get timezone-aware timestamps.
- Convert between UTC and local time using `DateTimeZone.ToLocal` and `DateTimeZone.ToUtc`.
- Remove a timezone offset using `DateTimeZone.RemoveZone`.
- Switch a value to a different timezone offset using `DateTimeZone.SwitchZone`.
- Apply a recommended timezone handling strategy for Power BI Service deployments.

Why Time Zones Matter in Power BI

When a Power BI dataset is refreshed in the Power BI Service, the refresh runs on Microsoft's cloud infrastructure in a specific data centre timezone — typically UTC. If your source data contains timestamps in local time (e.g. US Eastern time) and your M queries use `DateTime.LocalNow()` to determine 'today', the refresh result will be different depending on where and when it runs.

The safest approach for global deployments is: **always store and compare timestamps in UTC** within the data model, and convert to the user's local time only at the display layer (in DAX or report visuals).

WARNING Power BI Service scheduled refresh runs in UTC. If your report shows 'today's sales' using `DateTime.LocalNow()` and your business is in UTC-5, the refresh at 2:00 AM UTC will show data for the new UTC day while your users still consider it yesterday. Use `DateTimeZone.UtcNow()` and convert explicitly to avoid this.

Core DateTimeZone Functions

`DateTimeZone.UtcNow`

Syntax`DateTimeZone.UtcNow() as datetimetimezone`

Parameter	Type	Description
-----------	------	-------------

Returns	Returns the current UTC date and time as a datetimezone value with offset +00:00.
Example	<code>DateTimeZone.UtcNow()</code>
Result	#datetimezone(2024, 3, 15, 13, 30, 0, 0, 0) (UTC)
NOTE Use <code>DateTimeZone.UtcNow()</code> in Power BI Service-deployed models for consistent 'now' behaviour regardless of the data centre's local time.	

DateTimeZone.LocalNow

Syntax	DateTimeZone.LocalNow() as datetimezone		
Parameter	Type	Description	
Returns	Returns the current local date and time as a datetimezone value with the local system's UTC offset.		
Example	DateTimeZone.LocalNow()		
Result	#datetimezone(2024, 3, 15, 14, 30, 0, 1, 0) (UTC+01:00)		
NOTE	The offset reflects the timezone of the machine executing the M query. In Power BI Service this is UTC. In Power BI Desktop it is the developer's local timezone.		

DateTimeZone.ToUtc

Syntax	DateTimeZone.ToUtc(dateTimeZone as datetimezone) as datetimezone	
Parameter	Type	Description
dateTimeZone	datetimezone	A datetimezone value to convert to UTC.
Returns	Converts a datetimezone value to the equivalent UTC time (offset +00:00).	
Example	DateTimeZone.ToUtc(#datetimezone(2024, 3, 15, 14, 30, 0, 1, 0))	
Result	#datetimezone(2024, 3, 15, 13, 30, 0, 0, 0) (subtracted 1 hour)	
NOTE	DateTimeZone.ToUtc adjusts the time component while preserving the same instant. 14:30 UTC+1 = 13:30 UTC. The moment in time is identical, only the representation changes.	

DateTimeZone.ToLocal

Syntax	<code>DateTimeZone.ToLocal(dateTimeZone as datetimezone) as datetimezone</code>	
Parameter	Type	Description
<code>dateTimeZone</code>	<i>datetimezone</i>	A datetimezone value to convert to local time.
Returns	Converts a datetimezone value to the local system timezone.	

Example	<code>DateTimeZone.ToLocal(#datetimezone(2024, 3, 15, 13, 30, 0, 0, 0))</code>
Result	<code>#datetimezone(2024, 3, 15, 14, 30, 0, 1, 0)</code> (if local is UTC+1)

NOTE The result depends on the machine's local timezone. In Power BI Service, `ToLocal` returns UTC (no change), making it useless for user-timezone conversion. Use `DateTimeZone.SwitchZone` for explicit timezone conversion in the Service.

DateTimeZone.RemoveZone

Syntax	<code>DateTimeZone.RemoveZone(dateTimeZone as datetimezone) as datetime</code>
---------------	--

Parameter	Type	Description
<code>dateTimeZone</code>	<i>datetimezone</i>	A datetimezone value.
Returns	Strips the timezone offset and returns a datetime with the same date and time values.	
Example	<code>DateTimeZone.RemoveZone(#datetimezone(2024, 3, 15, 14, 30, 0, 1, 0))</code>	
Result	<code>#datetime(2024, 3, 15, 14, 30, 0)</code>	

NOTE `RemoveZone` does NOT convert the time — it simply removes the offset annotation. 14:30 UTC+1 with `RemoveZone` = 14:30 (no timezone). If you want UTC before removing: `DateTimeZone.RemoveZone(DateTimeZone.ToUtc(value))`.

DateTimeZone.SwitchZone

Syntax	<code>DateTimeZone.SwitchZone(dateTimeZone as datetimezone, timezoneHours as number, optional timezoneMinutes as nullable number) as datetimezone</code>
---------------	--

Parameter	Type	Description
<code>dateTimeZone</code>	<i>datetimezone</i>	The source datetimezone value.
<code>timezoneHours</code>	<i>number</i>	The target UTC offset in hours (e.g. -5 for US Eastern Standard Time).
<code>timezoneMinutes</code>	<i>nullable number</i>	Optional. Additional minutes for offsets like UTC+5:30 (India). Pass 30.
Returns	Converts a datetimezone value to the specified UTC offset, adjusting the time accordingly.	
Example	<code>DateTimeZone.SwitchZone(#datetimezone(2024, 3, 15, 13, 30, 0, 0, 0), -5)</code>	
Result	<code>#datetimezone(2024, 3, 15, 8, 30, 0, -5, 0)</code> (EST)	

NOTE `SwitchZone` is the function to use in Power BI Service when you need to convert UTC to a specific timezone. Pass the UTC offset: -5 for US Eastern (Standard), +5.5 requires (5, 30) as separate hour and minute arguments.

DateTimeZone.ZoneHours	Extracts the hours component of the UTC offset from a datetimezone value.
DateTimeZone.ZoneHours(datetimezone)	DateTimeZone.ZoneHours(#datetimezone(2024, 3, 15, 14, 30, 0, 5, 30)) → 5
DateTimeZone.ZoneMinutes	Extracts the minutes component of the UTC offset. Useful for offsets like UTC+5:30 (India) and UTC+5:45 (Nepal).
DateTimeZone.ZoneMinutes(datetimezone)	DateTimeZone.ZoneMinutes(#datetimezone(2024, 3, 15, 14, 30, 0, 5, 30)) → 30
DateTimeZone.From	Parses a datetimezone from text (ISO 8601 with offset) or converts from datetime.
DateTimeZone.From(value as any) as nullable datetimezone	DateTimeZone.From("2024-03-15T14:30:00+01:00") → #datetimezone(2024, 3, 15, 14, 30, 0, 1, 0)

Recommended Timezone Strategy for Power BI

Follow this four-step strategy for timezone-safe models:

- 1. Store UTC in the model.** Convert all source timestamps to UTC in M using DateTimeZone.ToUtc, then DateTimeZone.RemoveZone to get a clean datetime in UTC without the offset annotation.
- 2. Add a UTC offset column.** Add a separate integer column for the source timezone offset (e.g. UTCOffsetHours). This preserves the original context without complicating the model.
- 3. Convert at display time in DAX.** Create a DAX measure or calculated column that adds the user's local offset: LocalTime = [UTC_DateTime] + TIME(UTCOffsetHours, 0, 0).
- 4. Use DateTimeZone.UtcNow() for 'now'.** Replace DateTime.LocalNow() with DateTimeZone.RemoveZone(DateTimeZone.UtcNow()) to get a stable UTC 'now' in all environments.

LESSON 5

Duration Functions

After completing this lesson, you will be able to:

- Create duration values using the #duration literal and Duration.From.
- Extract days, hours, minutes, and seconds components from a duration.
- Use TotalDays, TotalHours, TotalMinutes, and TotalSeconds for scalar calculations.
- Add and subtract durations and dates to compute elapsed time.
- Build a business-day calculation using conditional logic and Duration.

The Duration Type

A duration represents a **span of time**, not a point in time. It is the result of subtracting one date or datetime from another, or can be constructed directly. Duration values support arithmetic — you can add a duration to a date, or divide a duration by a number:

Duration arithmetic:

```
// Construct a duration
OneDay = #duration(1, 0, 0, 0) // 1 day
NinetyMins = #duration(0, 1, 30, 0) // 90 minutes
TwoWeeks = #duration(14, 0, 0, 0) // 14 days

// Subtract two dates to get a duration
OrderDate = #date(2024, 3, 1),
ShipDate = #date(2024, 3, 8),
LeadTime = ShipDate - OrderDate // = #duration(7, 0, 0, 0)

// Add a duration to a date
DueDate = OrderDate + #duration(30, 0, 0, 0) // 30 days from order date

// Subtract a duration
PreviousWeek = #date(2024, 3, 15) - #duration(7, 0, 0, 0)
// = #date(2024, 3, 8)
```

Extracting Duration Components

Duration.Days

Syntax	Duration.Days(duration as nullable duration) as nullable number
--------	---

Parameter	Type	Description
duration	nullable duration	The duration value to inspect.
Returns	Returns the whole-day component of the duration (not the total number of days).	
Example	Duration.Days(#duration(7, 14, 30, 0))	
Result	7	

NOTE Duration.Days returns only the integer days part — NOT the total days including fractional days from hours/minutes. #duration(1, 12, 0, 0) has Duration.Days = 1, but Duration.TotalDays = 1.5. Use Duration.TotalDays for calculations.

Duration.TotalDays

Syntax	Duration.TotalDays(duration as nullable duration) as nullable number	
Parameter	Type	Description
duration	nullable duration	The duration value.
Returns	Returns the total duration expressed as a fractional number of days.	
Example	Duration.TotalDays(#duration(1, 12, 0, 0))	
Result	1.5	
NOTE Use Duration.TotalDays for calculations like average lead time, SLA compliance, and ageing analysis. Number.Round(Duration.TotalDays(ShipDate - OrderDate), 1) gives lead time in days with one decimal.		

Duration.Hours	Returns the hours component (0-23). For total hours including days: <code>Duration.TotalHours</code> .
<code>Duration.Hours(duration)</code>	<code>Duration.Hours(#duration(1, 14, 30, 0))</code> → 14
Duration.Minutes	Returns the minutes component (0-59). For total minutes: <code>Duration.TotalMinutes</code> .
<code>Duration.Minutes(duration)</code>	<code>Duration.Minutes(#duration(0, 1, 45, 30))</code> → 45
Duration.Seconds	Returns the seconds component including fractional seconds. For total: <code>Duration.TotalSeconds</code> .
<code>Duration.Seconds(duration)</code>	<code>Duration.Seconds(#duration(0, 0, 5, 30.5))</code> → 30.5
Duration.TotalHours	Total duration expressed as fractional hours. 1 day 6 hours = 30 total hours.
<code>Duration.TotalHours(duration)</code>	<code>Duration.TotalHours(#duration(1, 6, 0, 0))</code> → 30

Duration.TotalMinutes	Total duration as fractional minutes. 2 hours 30 minutes = 150 total minutes.
Duration.TotalMinutes(duration)	Duration.TotalMinutes(#duration(0, 2, 30, 0)) → 150
Duration.TotalSeconds	Total duration as fractional seconds. 5 minutes 30 seconds = 330 total seconds.
Duration.TotalSeconds(duration)	Duration.TotalSeconds(#duration(0, 0, 5, 30)) → 330
Duration.From	Converts a number (days) or text to a duration. Duration.From(1.5) = 1 day 12 hours.
Duration.From(value as any) as nullable duration	Duration.From(7) → #duration(7, 0, 0, 0)
Duration.ToRecord	Converts a duration to a record with Days, Hours, Minutes, Seconds fields.
Duration.ToRecord(duration)	Duration.ToRecord(#duration(1, 14, 30, 0)) → [Days=1, Hours=14, Minutes=30, Seconds=0]

Computing Business Days

A frequently requested calculation is the number of **working days** between two dates — excluding weekends and optionally public holidays. M does not have a built-in business day function, but it can be constructed using List.Dates, List.Select, and Date.DayOfWeek:

Business day calculation function:

```
// Function: count working days between two dates (Mon-Fri only)
CountWorkingDays = (startDate as date, endDate as date) as number =>
let
    // Generate all dates in the range
    DayCount = Duration.Days(endDate - startDate) + 1,
    AllDates = List.Dates(startDate, DayCount, #duration(1, 0, 0, 0)),

    // Keep only Mon-Fri (DayOfWeek Mon=1...Fri=5 when using Day.Monday)
    WorkDays = List.Select(AllDates, each
        Date.DayOfWeek(_, Day.Monday) < 5 // Mon=0, Tue=1, ..., Fri=4, Sat=5, Sun=6
    )
in
    List.Count(WorkDays),

// Usage in a Table.AddColumn:
AddLeadDays = Table.AddColumn(Source, "WorkingDayLeadTime",
    each CountWorkingDays([OrderDate], [ShipDate]),
    Int64.Type)
```

TIP To exclude public holidays, add a second filter in List.Select: each Date.DayOfWeek(_, Day.Monday) < 5 and not List.Contains(HolidayList, _). HolidayList is a list of date values loaded from a holiday calendar table. This pattern is covered in detail in Module 6 (List Functions).

LESSON 6

Building a Production Date Dimension Table

After completing this lesson, you will be able to:

- Explain why Power BI requires a separate, marked Date dimension table.
- Generate a contiguous date list using List.Dates.
- Add all standard calendar columns to the Date table.
- Add fiscal calendar columns with a configurable fiscal year start month.
- Add relative date columns (IsCurrentMonth, IsPriorYear, etc.).
- Mark the table as a Date Table in Power BI and verify time intelligence works.

Why a Separate Date Table Is Mandatory

Power BI includes an **Auto date/time** feature that automatically creates hidden date tables for every date column. This is convenient for beginners but has significant limitations in professional models:

- Auto date/time creates a **separate hidden table for every date column** in the model. With 10 date columns, you have 10 hidden tables — all consuming memory.
- Auto date/time tables cannot be shared across columns. A single DimDate shared by all date columns is far more efficient.
- Auto date/time tables use the default calendar (January fiscal year start). A custom Date table can implement any fiscal calendar.
- DAX time intelligence functions work identically whether you use auto or custom Date tables — but custom tables give you complete control.
- Auto date/time cannot add custom columns like IsWeekend, IsHoliday, FiscalYear, or RelativePeriod flags.

WARNING Turn off Auto date/time for professional models: File → Options → Data Load → uncheck 'Auto date/time for new files'. Then build your own Date table using the pattern in this lesson.

Generating Dates with List.Dates

The foundation of every M-built Date table is **List.Dates**, which generates a contiguous sequence of date values:

List.Dates

Syntax	List.Dates(start as date, count as number, step as duration) as list	
Parameter	Type	Description
start	date	The first date in the sequence.
count	number	The total number of dates to generate.
step	duration	The interval between each date. Use #duration(1,0,0,0) for daily.
Returns	Returns a list of date values starting at start, with count elements, each separated by step.	
Example	List.Dates(#date(2024,1,1), 5, #duration(1,0,0,0))	
Result	{#date(2024,1,1), #date(2024,1,2), #date(2024,1,3), #date(2024,1,4), #date(2024,1,5)}	
NOTE	Use Duration.Days(EndDate - StartDate) + 1 to compute the count automatically from start and end dates. This makes the date range dynamic and easy to configure.	

Complete Date Dimension Table — Full M Code

The following is a complete, production-ready Date dimension table query. It includes standard calendar columns, fiscal calendar columns (configurable), relative period flags, and a sort column for month names. Copy this directly into a blank query in Power Query:

Complete Date Dimension Table M Code:

[illegible]

[illegible]

[illegible]

WARNING After loading this query, mark it as the Date Table in Power BI Desktop: select the table in Model View → Table Tools → Mark as date table → select the 'Date' column. This enables all DAX time intelligence functions. The Date column must have no duplicates, no nulls, and a contiguous daily range.

Column Descriptions for the Date Dimension

The following table describes the purpose of each column in the Date dimension, helping you choose the right column for each DAX or report use case:

Column	Type	Purpose
Date	type date	The primary key. Connect all fact table date columns to this.

Year	Int64	Four-digit year integer. Use for year-level filtering and relationships.
YearLabel	text	Year as text (e.g. '2024'). Use for axis labels in visuals.
Quarter	Int64	Quarter number 1-4. Use for sorting QuarterLabel.
QuarterLabel	text	Quarter as text (e.g. 'Q3'). Use for axis labels.
YearQuarter	text	Combined (e.g. '2024 Q3'). Use for multi-year quarter axes.
MonthNum	Int64	Month number 1-12. ALWAYS use this to sort MonthName in Power BI visuals.
MonthName	text	Full month name (e.g. 'March'). Sort by MonthNum.
MonthShort	text	3-letter month abbreviation (e.g. 'Mar'). Sort by MonthNum.
YearMonth	Int64	YYYYMM integer (e.g. 202403). Sorts MonthName across years correctly.
DayNum	Int64	Day of month 1-31.
DayOfWeekNum	Int64	0=Monday ... 6=Sunday (ISO). Use to sort DayOfWeekName.
DayOfWeekName	text	Full weekday name. Sort by DayOfWeekNum.
DayOfWeekShort	text	3-letter weekday abbreviation (e.g. 'Mon').
WeekOfYear	Int64	ISO week number 1-53. Use for weekly trend charts.
DayOfYear	Int64	Day number within the year 1-366. Use for YTD comparisons.
IsWeekend	logical	True for Saturday and Sunday. Use for weekend exclusion filters.
IsWeekday	logical	True for Monday-Friday. Complement to IsWeekend.
FiscalYear	Int64	Fiscal year integer. FY2025 = July 2024 - June 2025 (with FiscalYearStart=7).
FiscalQuarter	Int64	Fiscal quarter 1-4 relative to FiscalYearStart.
FiscalMonth	Int64	Fiscal month 1-12 relative to FiscalYearStart.
FiscalYearLabel	text	Fiscal year as text (e.g. 'FY2025').
IsToday	logical	True only for today's date. Refreshed on each model refresh.
IsCurrentWeek	logical	True for all dates in the current calendar week.
IsCurrentMonth	logical	True for all dates in the current calendar month.
IsCurrentQuarter	logical	True for all dates in the current calendar quarter.
IsCurrentYear	logical	True for all dates in the current calendar year.
IsPriorMonth	logical	True for all dates in the previous calendar month.
IsPriorYear	logical	True for all dates in the previous calendar year.

DEMONSTRATION 1**Parsing and Standardising Mixed Date Formats**

In this demonstration, the instructor shows how to handle a common real-world scenario: a source system exports dates in multiple formats — some as ISO 8601 text, some as US locale text, some as Excel serial numbers, and some as datetimezone strings. All must be normalised to a single type date column.

Scenario: The GlobalSales.csv file contains an OrderTimestamp column with mixed formats in the same column. You must detect the format and parse correctly.

Demonstration Steps**Step 1: Load GlobalSales.csv and inspect the OrderTimestamp column**

Open the file in Power Query. Observe the Column Quality — some values are valid text, some are numbers, and some contain timezone offsets. Column profile shows min/max text lengths vary widely, indicating mixed formats.

Step 2: Open the Advanced Editor

Examine the M code. The OrderTimestamp column currently has type any. Attempting Table.TransformColumnTypes with type date will fail immediately.

Step 3: Build a safe date parsing function

Create a helper function ParseDate that uses try...otherwise to attempt multiple formats: first try Date.From(value), then try Date.From(DateTime.From(value)), then try Date.From(Number.From(value)) for Excel serials, finally return null on failure.

Step 4: Apply the function with Table.AddColumn

Add a ParsedDate column by calling ParseDate on [OrderTimestamp]. Inspect the result — most values should parse. Any remaining nulls indicate an unrecognised format that needs investigation.

Step 5: Handle the datetimezone format

Add a step that applies DateTimeZone.From on values containing a '+' sign, then DateTimeZone.RemoveZone(DateTimeZone.ToUtc(...)) to normalise to UTC date.

Step 6: Validate: remove nulls and check row count

Filter out null ParsedDate rows and compare the count to the original. Any missing rows represent formats that need additional handling.

Step 7: Finalise: remove the original text column

After validating ParsedDate, remove the original OrderTimestamp column and rename ParsedDate to OrderDate.

RESULT GlobalSales has an OrderDate column of type date with no nulls (assuming all formats are handled). Mixed-format sources are now normalised to a single, consistently typed date column ready for the Date dimension relationship.

DEMONSTRATION 2

Building a Complete Date Dimension from Scratch

In this demonstration, the instructor pastes the complete Date dimension table code from Lesson 6 into a blank query and walks through each section, explaining the fiscal calendar logic and the relative period flag calculations.

Scenario: The company's fiscal year starts in July. Build a Date table spanning 2015–2030 with fiscal calendar columns for FY reporting.

Demonstration Steps

Step 1: Create a new blank query named DimDate

In Power Query Editor: Home → New Source → Blank Query. Rename to DimDate in the Query Settings pane.

Step 2: Paste the complete Date dimension code

Open the Advanced Editor. Delete 'in null'. Paste the full Date dimension code from Lesson 6. Change FiscalYearStart to 7 (July). Click Done.

Step 3: Walk through the Configuration section

Explain: StartDate, EndDate, FiscalYearStart, and Culture are the four knobs to adjust for any deployment. Changing FiscalYearStart = 1 produces a calendar year fiscal model.

Step 4: Explain the List.Dates generation

Show how $\text{DayCount} = \text{Duration.Days}(\text{EndDate} - \text{StartDate}) + 1$ dynamically computes the row count. Changing EndDate automatically generates more or fewer rows. No hard-coding needed.

Step 5: Walk through the fiscal calendar logic

Explain the FiscalYear calculation: months from July onwards belong to the next FY. Show FiscalMonth using Number.Mod to re-sequence July=1, August=2, ..., June=12.

Step 6: Show the relative date flags

Demonstrate that IsCurrentMonth, IsToday, IsPriorYear update automatically on each refresh because they reference DateTime.FixedLocalNow(). No parameter changes needed.

Step 7: Verify the output

Check: 5,844 rows (2015–2030), no duplicate dates, no nulls in Date column. Sort by FiscalYear and verify July starts a new FY.

Step 8: Mark as Date Table in Power BI

Close & Apply. In Model View, right-click DimDate → Mark as date table → select Date column. Verify the confirmation dialog shows no errors.

RESULT DimDate is loaded with 5,844 rows, all date functions correctly computed, fiscal calendar columns reflecting the July fiscal year start, and relative period flags that refresh automatically. The table is marked as the Date Table in Power BI, enabling full DAX time intelligence.

LAB 3

Dates, Times, and Durations

In this lab you will apply all date, time, duration, and timezone functions from Module 3. You will enrich the FactSales query with date-derived columns, compute lead times and ageing durations, handle a global timezone scenario, and build and load a complete Date dimension table.

Lab Scenario

Contoso Analytics has expanded globally. Their sales system records Order Date, Ship Date, and a UTC timestamp for each transaction. Your task is to: add full date intelligence columns to the fact table, compute lead time in working days, convert UTC timestamps to the company's reporting timezone (US Eastern, UTC-5), and build a shared Date dimension covering the company's fiscal calendar (fiscal year starts October 1).

NOTE This lab requires the GlobalSales.csv file from the companion materials. Also continue with the FactSales query from Labs 1 and 2. If FactSales has Order Date and Ship Date columns, you are ready to begin.

Exercise 1: Extract Date Components and Build Calendar Columns (Estimated time: 20 minutes)

In this exercise you add a full set of calendar columns to the FactSales query.

1. Select the **FactSales** query. Verify Order Date is type date.
2. Add a column **OrderYear** using **Date.Year**. Type: Int64.Type.
3. Add a column **OrderMonth** using **Date.Month**. Type: Int64.Type.
4. Add a column **OrderMonthName** using **Date.MonthName** with culture 'en-US'. Type: type text.
5. Add a column **OrderQuarter** using **Date.QuarterOfYear**. Type: Int64.Type.
6. Add a column **OrderQuarterLabel** combining 'Q' with **Text.From(Date.QuarterOfYear([Order Date]))**. Type: type text.
7. Add a column **OrderDayOfWeek** using **Date.DayOfWeekName** with culture 'en-US'. Type: type text.
8. Add a column **IsWeekendOrder** (logical) that is true when DayOfWeek for Order Date is Saturday or Sunday. Use **Date.DayOfWeek** with **Day.Monday**: value ≥ 5 means weekend.
9. Add a column **OrderStartOfMonth** using **Date.StartOfMonth** on [Order Date]. Type: type date. This is useful for monthly aggregations.
10. Add a column **DaysToEndOfMonth**: the number of days between Order Date and **Date.EndOfMonth([Order Date])**. Use **Duration.Days** with subtraction.

11. Verify column types. Remove any unwanted columns. Check column quality shows 100% valid for all new date columns.

RESULT FactSales has OrderYear, OrderMonth, OrderMonthName, OrderQuarter, OrderQuarterLabel, OrderDayOfWeek, IsWeekendOrder, OrderStartOfMonth, and DaysToEndOfMonth columns. All are correctly typed. IsWeekendOrder is true for Saturday and Sunday orders only.

Exercise 2: Duration Calculations — Lead Times and Ageing (Estimated time: 20 minutes)

In this exercise you compute duration-based business metrics.

1. Verify FactSales has both **Order Date** (type date) and **Ship Date** (type date). If Ship Date is missing, create it by adding a random 2-7 day offset to Order Date for testing.
2. Add a column **LeadTimeDays**: the total days between Order Date and Ship Date using **Duration.TotalDays**([Ship Date] - [Order Date]). Round to 0 decimal places. Type: Int64.Type.
3. Add a column **LeadTimeCategory** using nested if: 'Express' (1-2 days), 'Standard' (3-5 days), 'Delayed' (6-10 days), 'Critical' (>10 days), 'Error' if LeadTimeDays < 0.
4. Add a column **WorkingDayLeadTime** using the CountWorkingDays function pattern from Lesson 5. This requires defining the function as a local step inside the let block.
5. Add a column **DaysSinceOrder**: days from Order Date to today. Use **DateTime.Date(DateTime.FixedLocalNow())** - [Order Date], then **Duration.TotalDays**. Round to 0 decimals.
6. Add a column **AgeingBucket**: classify DaysSinceOrder as '0-30 days', '31-60 days', '61-90 days', 'Over 90 days'. Type: type text.
7. Verify LeadTimeDays has no negative values (Ship Date should never be before Order Date). If any exist, add a data quality flag column **IsLeadTimeError** (logical).

RESULT FactSales has LeadTimeDays (integer, 0+), LeadTimeCategory (text, 4 categories), WorkingDayLeadTime (integer, business days only), DaysSinceOrder (integer), and AgeingBucket (text, 4 buckets). No negative lead times. IsLeadTimeError = false for all rows if data is clean.

Exercise 3: Time Zone Handling for a Global Dataset (Estimated time: 20 minutes)

In this exercise you load GlobalSales.csv and normalise its UTC timestamps to US Eastern time for reporting.

1. Load **GlobalSales.csv** as a new query named GlobalSales. This file contains a UTCTimestamp column with datetimezone values in ISO 8601 format.
2. Inspect the UTCTimestamp column. Promote headers and verify the raw values look like: '2024-03-15T14:30:00+00:00'.

3. Add a step: **ParsedUTC = Table.TransformColumnTypes(Headers, {"UTCTimestamp", type datetimezone})**. Verify no errors appear.
4. Add a column **UTCDateTime**: use **DateTimeZone.RemoveZone(DateTimeZone.ToUtc([UTCTimestamp]))** to get a clean UTC datetime without offset. Type: type datetime.
5. Add a column **EasternTime**: convert UTCimestamp to US Eastern Standard Time (UTC-5) using **DateTimeZone.SwitchZone([UTCTimestamp], -5)**. Type: type datetimezone.
6. Add a column **EasternDate**: extract just the date from EasternTime using **DateTime.Date(DateTimeZone.RemoveZone([EasternTime]))**. Type: type date.
7. Add a column **EasternHour**: extract the hour from EasternTime using **Time.Hour(DateTimeZone.RemoveZone([EasternTime]))**. Type: Int64.Type.
8. Add a column **TimeOfDayBucket**: classify EasternHour as 'Morning' (6-11), 'Afternoon' (12-17), 'Evening' (18-21), 'Night' (all others).
9. Verify: rows with UTCimestamp = midnight UTC (00:00) should show EasternDate = previous day (due to UTC-5 offset).
10. Rename the query to **GlobalFactSales** and set Enable load = true.

RESULT GlobalFactSales has UTCDateTime (datetime, UTC), EasternTime (datetimezone, UTC-5), EasternDate (date), EasternHour (int), and TimeOfDayBucket (text). Midnight UTC rows correctly show the previous day's Eastern date. TimeOfDayBucket distributes orders across Morning, Afternoon, Evening, and Night.

Exercise 4: Build and Load a Complete Date Dimension Table (Estimated time: 25 minutes)

In this exercise you build the full Date dimension with Contoso's fiscal calendar (October start) and load it as the official Date table for the model.

1. Create a new blank query. Name it **DimDate**.
2. Open the Advanced Editor. Paste the complete Date dimension code from Lesson 6. Set **FiscalYearStart = 10** (October). Set StartDate = 1 January 2015, EndDate = 31 December 2030.
3. Click Done. Verify the data preview shows 5,844 rows with the Date column starting 2015-01-01.
4. Verify FiscalYear: for a row with Date = 2024-10-15, FiscalYear should be 2025 (since FY2025 starts October 2024).
5. Verify FiscalMonth: for Date = 2024-10-15, FiscalMonth should be 1 (first month of the fiscal year).
6. Verify FiscalMonth for Date = 2024-09-30: FiscalMonth should be 12 (last month of FY2024).

7. Verify `IsCurrentMonth`: filter to see only rows where `IsCurrentMonth = true`. The count should equal the number of days in the current calendar month.
8. Verify `IsWeekend`: filter to `IsWeekend = true` and check that only Saturdays and Sundays appear in the `DayOfWeekName` column.
9. Click **Close & Apply** to load all queries.
10. In Power BI Desktop **Model View**: right-click `DimDate` → Mark as date table → select the `Date` column. Confirm the dialog shows no validation errors.
11. Create a relationship: `DimDate[Date]` → `FactSales[Order Date]` (Many-to-One). Verify the relationship line appears with the correct cardinality symbol.
12. In Report View, create a table visual with `DimDate[MonthName]` and a SUM of `FactSales[Sales]`. Verify the months appear and the totals are correct.
13. **Bonus:** Create a simple DAX measure: `YTD Sales = TOTALYTD(SUM(FactSales[Sales]), DimDate[Date])`. Add to the visual. Verify it shows cumulative values within each year.

RESULT DimDate is loaded with 5,844 rows covering 2015-2030. FiscalYear columns correctly reflect an October fiscal year start. DimDate is marked as the Date Table in Power BI. A Many-to-One relationship connects FactSales to DimDate via Order Date. The table visual shows sales by month. The TOTALYTD measure works correctly, confirming that time intelligence is fully functional.

Module Review and Takeaways

Review Questions

What is the difference between type date and type datetime in M?

Answer: type date represents a calendar day only (no time component). type datetime includes both date and time. Use type date for pure date columns like Order Date or Birth Date. Use type datetime for timestamps that include clock time.

What function would you use to get the last day of the month for a given date?

Answer: Date.EndOfMonth(dateValue). It automatically handles varying month lengths including leap years: Date.EndOfMonth(#date(2024, 2, 1)) = #date(2024, 2, 29).

What is the difference between Duration.Days and Duration.TotalDays?

Answer: Duration.Days returns only the integer days component of the duration — for #duration(1, 12, 0, 0) it returns 1. Duration.TotalDays returns the complete duration expressed as a fractional number of days — for #duration(1, 12, 0, 0) it returns 1.5. Use Duration.TotalDays for calculations.

Why should you use DateTime.FixedLocalNow() instead of DateTime.LocalNow() in most queries?

Answer: DateTime.LocalNow() is re-evaluated on each call and can return slightly different values within the same query. DateTime.FixedLocalNow() returns the same timestamp for the entire query evaluation, ensuring consistent comparisons across multiple steps.

What does DateTimeZone.RemoveZone do, and when should you use it?

Answer: It strips the timezone offset annotation from a datetimezone value and returns a datetime with the same date and time values (it does NOT convert the time). Use it after DateTimeZone.ToUtc() to get a clean UTC datetime without the +00:00 annotation, which is often required for model relationships to date columns.

Why must a Date table be 'marked as Date Table' in Power BI?

Answer: Marking enables DAX time intelligence functions (TOTALYTD, SAMEPERIODLASTYEAR, etc.). Without this marking, these functions do not recognise the table as a proper calendar and will produce incorrect or error results. The Date column must have no nulls, no duplicates, and a contiguous daily range.

How do you calculate the number of working days between two dates in M?

Answer: Generate a list of all dates in the range using List.Dates, then filter to keep only Mon-Fri using List.Select with Date.DayOfWeek(_, Day.Monday) < 5, then count the remaining dates with List.Count.

What is the purpose of the YearMonth (YYYYMM) column in the Date dimension?

*Answer: It provides a numeric sort key for the MonthName column in Power BI visuals. MonthName is text and sorts alphabetically (April, August...) without a numeric sort column. YearMonth = Year * 100 + Month gives unique, chronologically ordered integers that Power BI can use to sort MonthName correctly across multiple years.*

Best Practices

- Always specify a culture parameter when using `Date.MonthName` and `Date.DayOfWeekName`. Without it, the result depends on the machine's locale, producing different results on different systems.
- Use `Date.EndOfMonth` rather than calculating month-end manually. Hard-coding month lengths (28, 30, 31) fails for February and leap years.
- Never store dates as text strings in the data model. Text dates cannot participate in date table relationships or DAX time intelligence calculations.
- Use `DateTime.FixedLocalNow()` and assign it to a named step at the top of your query. Reference that step throughout rather than calling `DateTime.LocalNow()` multiple times.
- For global Power BI Service deployments, use `DateTimeZone.UtcNow()` for all 'current time' logic. `DateTime.LocalNow()` returns UTC in the Service, which may differ from your developer machine.
- Build one shared `DimDate` table and connect all date columns in all fact tables to it. Never use the auto date/time feature in professional models.
- In the Date dimension, always include a numeric sort column (`YearMonth = Year*100 + Month`) to enable correct chronological sorting of `MonthName` in Power BI visuals.
- Turn off Auto date/time in Power BI Desktop Options before building a professional model. Multiple hidden date tables consume unnecessary memory.
- Use `Duration.TotalDays` (not `Duration.Days`) when computing elapsed time for calculations like average lead time or SLA analysis.
- For fiscal calendar columns, use the `Number.Mod` pattern: `FiscalMonth = Number.Mod(CalendarMonth - FiscalYearStart + 12, 12) + 1`. This handles the year-wrap correctly for all fiscal year starts.

Common Issues and Solutions

Date column not connecting to Date Table

Cause: The fact table date column is type `datetime` but the `DimDate` Date column is type `date`. Power BI relationships require matching types.

Solution: Add a step in the fact table query: `OrderDate = Table.TransformColumnTypes(Source, {"Order Date", type date})`. Or use `DateTime.Date([OrderTimestamp])` to extract just the date portion.

Date.MonthName shows wrong language

Cause: No culture was specified, so M uses the machine's current locale. Different machines produce different month names.

Solution: Always pass a culture: `Date.MonthName([Date], "en-US")`. For international reports, store `MonthNum` as the sort column and let the report layer handle localisation.

TOTALYTD and other DAX time intelligence functions return blank

Cause: The Date table has not been marked as a Date Table, or the relationship to the fact table date column is missing or inactive.

Solution: In Power BI Model View: right-click DimDate → Mark as date table → select the Date column. Verify the relationship to the fact table is active (solid line, not dashed).

Duration calculation returns error on null dates

Cause: Subtracting null - date or date - null produces an error, not null.

Solution: Wrap in a null check: if [Ship Date] = null or [Order Date] = null then null else Duration.TotalDays([Ship Date] - [Order Date]).

FiscalYear is off by one

Cause: The fiscal year calculation adds 1 when the month $\geq$ FiscalYearStart, but the business definition uses a different convention.

Solution: Clarify the business convention: does FY2025 mean 'the year ending June 2025' (add 1 from July) or 'the year starting July 2025' (add 1 from August)? Adjust the $\geq$ comparison in the FiscalYear formula accordingly.

Key Functions — Module 3 Quick Reference

All date, time, datetime, datetimezone, and duration functions introduced in this module, organised by category for fast reference.

Function	Category	Returns / Key Behaviour
Date.Year	Date Extract	Year as integer
Date.Month	Date Extract	Month 1-12 as integer
Date.Day	Date Extract	Day of month 1-31
Date.DayOfWeek	Date Extract	0-6 (0 = firstDayOfWeek param)
Date.DayOfWeekName	Date Extract	Full weekday name (localised)
Date.DayOfYear	Date Extract	Day number in year 1-366
Date.WeekOfYear	Date Extract	ISO week number 1-53
Date.WeekOfMonth	Date Extract	Week number in month 1-5
Date.QuarterOfYear	Date Extract	Quarter 1-4
Date.MonthName	Date Extract	Full month name (localised)
Date.StartOfMonth	Date Boundary	First day of containing month
Date.EndOfMonth	Date Boundary	Last day of containing month (auto leap-year)
Date.StartOfQuarter	Date Boundary	First day of containing quarter
Date.EndOfQuarter	Date Boundary	Last day of containing quarter
Date.StartOfYear	Date Boundary	January 1 of containing year
Date.EndOfYear	Date Boundary	December 31 of containing year
Date.StartOfWeek	Date Boundary	First day of containing week
Date.EndOfWeek	Date Boundary	Last day of containing week
Date.AddDays	Date Arithmetic	Date +/- n days
Date.AddMonths	Date Arithmetic	Date +/- n months (handles month-end clamping)
Date.AddYears	Date Arithmetic	Date +/- n years
Date.AddQuarters	Date Arithmetic	Date +/- n quarters
Date.AddWeeks	Date Arithmetic	Date +/- n weeks
Date.IsInCurrentMonth	Date Relative	True if date is in current calendar month
Date.IsInCurrentYear	Date Relative	True if date is in current calendar year
Date.IsInPreviousMonth	Date Relative	True if date is in previous calendar month
Date.From	Date Convert	Convert text / datetime / number to date
Time.Hour	Time Extract	Hour component 0-23
Time.Minute	Time Extract	Minute component 0-59
Time.Second	Time Extract	Second component (with fractional)
Time.StartOfHour	Time Boundary	Time truncated to hour start (mm=00, ss=00)

DateTime.Date	DateTime	Extract date portion from datetime
DateTime.Time	DateTime	Extract time portion from datetime
DateTime.LocalNow	DateTime	Current local datetime (re-evaluated each call)
DateTime.FixedLocalNow	DateTime	Current local datetime (fixed for query lifetime)
DateTime.From	DateTime	Convert date / text / number to datetime
DateTime.IsInCurrentHour	DateTime Rel.	True if datetime is in current hour
DateTimeZone.UtcNow	DTZ	Current UTC time as datetimezone
DateTimeZone.LocalNow	DTZ	Current local time as datetimezone
DateTimeZone.ToUtc	DTZ Convert	Convert datetimezone to UTC offset
DateTimeZone.ToLocal	DTZ Convert	Convert datetimezone to local system timezone
DateTimeZone.RemoveZone	DTZ Convert	Strip offset — returns datetime (no conversion)
DateTimeZone.SwitchZone	DTZ Convert	Convert to specified UTC offset
DateTimeZone.ZoneHours	DTZ Extract	Hours component of the UTC offset
DateTimeZone.ZoneMinutes	DTZ Extract	Minutes component of the UTC offset
DateTimeZone.From	DTZ Convert	Parse text or convert to datetimezone
Duration.Days	Duration	Integer days component only
Duration.Hours	Duration	Integer hours component (0-23)
Duration.Minutes	Duration	Integer minutes component (0-59)
Duration.Seconds	Duration	Seconds component (with fractional)
Duration.TotalDays	Duration Total	Complete duration as fractional days
Duration.TotalHours	Duration Total	Complete duration as fractional hours
Duration.TotalMinutes	Duration Total	Complete duration as fractional minutes
Duration.TotalSeconds	Duration Total	Complete duration as fractional seconds
Duration.From	Duration	Convert number (days) or text to duration
Duration.ToRecord	Duration	Convert to record {Days, Hours, Minutes, Seconds}
List.Dates	List/Date	Generate contiguous list of dates (Date table foundation)

TIP The complete formula reference for all M functions across all 12 modules is in the companion document: [PowerQuery_M_Formula_Reference.pdf](#)

Module Summary

- M provides five temporal types: date (day only), time (clock time only), datetime (combined), datetimezone (with UTC offset), and duration (time interval). Choose based on what the source data represents.
- Date component functions (Date.Year, Date.Month, Date.Day, Date.QuarterOfYear, etc.) are the building blocks of every Date dimension table. Always specify a culture for name functions

(Date.MonthName, Date.DayOfWeekName).

- Boundary functions (Date.StartOfMonth, Date.EndOfMonth, etc.) return the first or last moment of a calendar period. They are null-safe and handle varying month lengths automatically.
- Duration arithmetic: subtracting two dates gives a duration; adding a duration to a date gives a new date. Use Duration.TotalDays (not Duration.Days) for numeric calculations.
- For global Power BI deployments, store timestamps in UTC (DateTimeZone.ToUtc + DateTimeZone.RemoveZone) and convert to display timezone in DAX or using DateTimeZone.SwitchZone in M.
- Every professional Power BI model requires a single, shared, marked Date dimension table with a contiguous daily range. Build it with List.Dates, mark it as the Date Table in Power BI, and connect all fact table date columns to it.
- Fiscal calendar columns require the Number.Mod pattern for correct month re-sequencing. Relative period flags (IsCurrentMonth, IsPriorYear, etc.) refresh automatically because they reference DateTime.FixedLocalNow().

Next Module

Module 4: List Functions — Covers all List.* functions: creation, transformation, filtering, aggregation, set operations, and advanced patterns including List.Accumulate and List.Generate for iterative computation.